Data Structure, Midterm

A. Single Choice Questions (5% each, 60%)

(1) Which of the following statements about algorithms and computability is **TRUE**?

(A) As long as a procedure eventually produces a result, its individual steps may be left open to interpretation and it still qualifies as an algorithm.

(B) A step-by-step procedure that correctly solves only one particular instance of a problem, but no other instance, is still an algorithm for that problem.

(C) A problem is *computable* if an algorithm exists that solves it, and every algorithm must terminate in a finite amount of time.

(D) Programming is the design of an algorithm, while encoding the algorithm into a programming language is called abstraction.

(2) Which of the following statements about access control and inheritance in C++ classes is **FALSE**?

(A) A derived (child) class cannot add data or methods of its own; it can only use what it inherits from its parent.

(B) Protected members behave like private members for outside code, but they remain visible to derived classes.

(C) The compiler prevents code outside the class from accessing private members, so that code must use the class's public methods.

(D) If a class gives no access level for its members, they are private by default.

(3) The exact number of steps of an algorithm is $T(n) = 5n^2 + 2n + 7$. A program running it takes 2 seconds for $n = 10{,}000$. Using the dominant term, about how long should it take for $n = 30{,}000$ on the same computer?

(A) About 54 seconds

(B) About 6 seconds

(C) About 12 seconds

(D) About 18 seconds

(4) Which of the following statements about algorithm analysis is **TRUE**?

(A) Because $5n^2 + 27n + 1005$ is dominated by the constant 1005 when n is small (say 1 or 2), the order of magnitude of this algorithm is $O(1)$.

(B) Benchmark timings depend on the machine and compiler; counting steps as a function of n characterizes the algorithm itself.

(C) An algorithm with exact step count $T(n) = n + 1$ is $O(n + 1)$, which is a different class from $O(n)$.

(D) The order of magnitude keeps the coefficient of the dominant term, so $3n^2 + 2n + 4$ is $O(3n^2)$ rather than $O(n^2)$.

(5) A loop variable starts at $i = n$. After each iteration, i is replaced by $\lfloor i/2 \rfloor$, and the loop runs while $i \geq 1$. For $n = 1000$, how many iterations run, and what is the order of magnitude of the loop?

(A) 9 iterations; $O(\log n)$

(B) 500 iterations; $O(n)$

(C) 10 iterations; $O(\log n)$

(D) 1000 iterations; $O(n)$

(6) Which of the following statements about the dynamic array (`ArrayList`) built in class is **FALSE**?

(A) To avoid overwriting data, the shifting in `insert(idx, val)` starts at the insertion point and proceeds toward the end of the array.

(B) `insert(idx, val)` shifts the elements at `idx` and beyond one position toward the end; the worst case is `idx = 0`, so insert is $O(n)$.

(C) `erase(idx)` shifts every element after `idx` one position to the left, which takes $n - 1 - idx$ moves, so erasing index 0 is the worst case.

(D) The dynamic array keeps extra capacity and remembers how many slots are used, so most appends need no new allocation.

(7) A 200×300 matrix has 1,800 non-zero entries. In class, a matrix is sparse when **NNZ / size < 0.05**. Which statement about this matrix and its COOrdinate (COO) format is **correct**?

(A) It is sparse; COO stores $2 \times 1{,}800$ numbers, one value and one position index per entry.

(B) It is not sparse, because NNZ / size $= 0.06$ exceeds 0.05.

(C) It is sparse; COO stores 1,800 numbers, because only the values are kept.

(D) It is sparse (NNZ / size $= 0.03$); COO stores $3 \times 1{,}800 = 5{,}400$ numbers.

(8) The `add(item)` operation of the unordered linked list creates a new node and then performs two steps. Which statement is **TRUE**?

1. set the new node's link to refer to the current first node;
2. make `head` refer to the new node.

(A) The order of the two steps does not matter, because the original nodes stay in memory and the list operations can still find them.

(B) If the two steps are reversed, the new node links to itself and the original nodes are lost, because `head` was their only external reference.

(C) If the two steps are reversed, only the new item is lost, because the new node never becomes part of the list; the original list is unaffected.

(D) If the two steps are reversed, the new node is attached after the last node, so the list stays complete but `add` behaves like `append`.

(9) Which of the following statements about the ordered linked list (ascending order) from class is **FALSE**?

(A) `search(item)` can return false as soon as it reaches a node whose value is greater than `item`.

(B) `remove(item)` for a value that is not in the list stops once it reaches a value greater than `item` and leaves the list unchanged.

(C) `add(item)` must first locate the correct insertion position, rather than always inserting at the head as the unordered list does.

(D) Because `search` can stop early, searching an ordered linked list takes $O(\log n)$ time.

(10) Which of the following statements about converting infix to postfix with the operator-stack algorithm from class is **TRUE**?

(A) When an operator is read, stacked operators of higher or equal precedence are output first, so `A * B + C` becomes `A B * C +`.

(B) When a right parenthesis is read, it is pushed onto the stack and is popped later, when the matching left parenthesis is read from the input.

(C) Operands are pushed onto the stack as well, and they are output only when an operator that uses them is popped from the stack.

(D) At the end of the input, the operators still on the stack are output in the same order in which they were pushed (bottom first).

(11) Which of the following statements about queues and the Hot Potato simulation from class is **TRUE**?

(A) After `num` passes the child at the rear holds the potato, so the rear item is removed.

(B) A stack would simulate the game equally well, since in every round the child who joined the line most recently is the one eliminated.

(C) Each pass dequeues the child at the front and enqueues that child at the rear; after `num` passes, the child at the front is removed.

(D) The simulation removes one child per round and ends only when the queue is empty; the last child removed is the winner.

(12) What is the value of the postfix expression

```
7 2 3 * - 8 2 / +
```

(A) 3

(B) 5

(C) 1.25

(D) −0.75

B. Short-answer questions, Please provide the derivation for each question along with your answer (8% each, 40%).

(13) Briefly explain each of the following terms.

(a) Procedural abstraction

(b) Worst-case performance

(c) Base address of an array

(d) Head of a linked list

(14-1) Evaluate the time complexity of the following code segment in terms of Big-O notation (Please report both the complexity of each `for` loop and the whole segment):

```
int x = 0;
for (int i = 1; i <= n; i++) {
    int j = n;
    while (j >= 1) {
        x++;
```

```
        j = j / 2;
    }
}

for (int k = 0; k < 2 * n; k++) {
    x--;
}
```

(14-2) Evaluate the time complexity of the following code segment in terms of Big-O notation (Please report both the complexity of each `for` loop and the whole segment):

```
x = 0
for i in range(1, n + 1):
    j = n
    while j >= 1:
        x += 1
        j = j // 2

for k in range(2 * n):
    x -= 1
```

(15) Convert the infix expression `( A + B ) * C - D / ( E - F )` to postfix using a stack, as described in class.

(a) Show the state of the operator stack (bottom → top) and the output after processing EACH token.

(b) What is the resulting postfix expression?

(c) What is the maximum number of symbols (operators and left parentheses) in the operator stack at any one moment?

(d) What is the equivalent prefix expression?

(16) An empty ordered linked list (ascending order, implemented as in class) performs `add(40)`, `add(15)`, `add(62)`, `add(27)`, `add(55)`. Then it performs `search(30)` and `search(70)`. For each operation, count the number of different nodes whose value is compared with the item (a node counts once even if it is compared more than once).

(a) Show the list after each `add`, and the number of nodes examined by each `add`.

(b) How many nodes do `search(30)` and `search(70)` examine, and what do they return?

(c) If the same five values were added, in the same order, to an **unordered** linked list from class, what would the list hold, and how many nodes would `search(70)` examine? What does this say about the advantage of the ordered list and about its worst case?

(17) A two-dimensional array `Y` is stored contiguously in memory on a computer whose storage order (row-major or column-major) is not known in advance (zero-based indices; every index mentioned below is valid). It is known that

- `Y[2][1]` is located at address **604**
- `Y[0][4]` is located at address **532**
- `Y[3][5]` is located at address **684**

(a) Is the storage order row-major or column-major? Justify your answer.

(b) What is the size of each element?

(c) How many columns does `Y` have?

(d) What is the address of `Y[4][2]`?

Bonus: Deque (10%)

A queue `q` holds, from front to rear, `3 8 1`. An empty deque `d` supports the Deque Abstract Data Type (ADT) operations `addFront`, `addRear`, `removeFront`, `removeRear` (the two remove operations return the removed item), `isEmpty`, and `size`. Two steps are carried out:

- **Step 1:** repeatedly dequeue the front item of `q` and add it to the **front** of `d` with `addFront`, until `q` is empty.
- **Step 2:** repeatedly remove the front item of `d` with `removeFront` and enqueue it into `q`, until `d` is empty.

(a) Show `d` (front → rear) after each `addFront` and `q` (front → rear) after each `enqueue`. What is the final content of `q`?

(b) Repeat (a) when Step 1 uses `addRear` instead of `addFront`. What is the final content of `q` now?

(c) Which pairs of deque operations, used alone, give stack (LIFO) behavior, and which give queue (FIFO) behavior?

(d) Suppose the deque is implemented as a **singly** linked list that keeps a pointer to its first node (front) and to its last node (rear). Give the Big-O of `addFront`, `addRear`, `removeFront`, `removeRear`, explain which one is not $O(1)$, and state what change to the nodes fixes it.